

姓名	李泽浩	学号	2025218895	实验成绩	
专业班级	软件工程 25-9 班	实验时间	4.19	实验地点	新安学堂

实验二：链表实验

1 实验目的

- 1) 熟练掌握线性表的链式存储结构。
- 2) 熟练掌握单链表的有关算法设计和实现。
- 3) 根据具体问题的需要，设计出合理的表示数据的链式存储结构，并设计相关算法。

2 实验要求

- 1) 本次实验中的链表结构指带头结点的单链表；
- 2) 单链表结构和运算定义，算法的实现以库文件方式实现，不得在测试主程序中直接实现；比如存储、算法实现放入文件：linkedList.h
- 3) 程序运行、测试正确；
- 4) 实验程序有较好可读性，各运算和变量的命名直观易懂，符合软件工程要求；
- 5) 程序有适当的注释，程序的书写要采用缩进格式。

3 实验任务

编写算法实现下列问题的求解。

- 1) 将单链表 L 中的奇数项和偶数项结点分解开（元素值为奇数、偶数），分别放入新的单链表中，然后原表和新表元素同时输出到屏幕上，以便对照求解结果。实验测试数据基本要求：
 第一组数据：单链表元素为 （1,2,3,4,5,6,7,8,9,10,20,30,40,50,60）
 第二组数据：单链表元素为 （10,20,30,40,50,60,70,80,90,100）

- 2) 求两个递增有序单链表 L1 和 L2 中的公共元素，放入新的单链表 L3 中。

实验测试数据基本要求：

第一组

第一个单链表元素为 （1, 3, 6, 10, 15, 16, 17, 18, 19, 20）

第二个单链表元素为 （1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 18, 20, 30）

第二组

第一个单链表元素为 （1, 3, 6, 10, 15, 16, 17, 18, 19, 20）

第二个单链表元素为 （2, 4, 5, 7, 8, 9, 12, 22）

第三组

第一个单链表元素为 （）

第二个单链表元素为 （1, 2, 3, 4, 5, 6, 7, 8, 9, 10）

- 3) 删除递增有序单链表中的重复元素，要求时间性能最好。

实验测试数据基本要求：

第一组数据：单链表元素为 （1,2,3,4,5,6,7,8,9）

第二组数据：单链表元素为 （1,1,2,2,2,3,4,5,5,5,6,6,7,7,8,8,9）

第三组数据：单链表元素为 （1,2,3,4,5,5,6,7,8,8,9,9,9,9,9）

- 4) 递增有序单链表 L1、L2，不申请新结点，利用原表结点对两表进行合并，并使得

合并后成为一个集合，合并后用 L1 的头结点作为头结点，删除多余的结点，删除 L2 的头结点。要求时间性能最好。实验测试数据基本要求：

第一组

第一个单链表元素为 (1, 3, 6, 10, 15, 16, 17, 18, 19, 20)

第二个单链表元素为 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 18, 20, 30)

第二组

第一个单链表元素为 (1, 3, 6, 10, 15, 16, 17, 18, 19, 20)

第二个单链表元素为 (2, 4, 5, 7, 8, 9, 12, 22)

第三组

第一个单链表元素为 ()

第二个单链表元素为 (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

- 5) 已知一个带有表头结点的单链表，结点结构如下图。假设该链表只给出了头指针 list。在不改变链表的前提下，请设计一个尽可能高效的算法，查找链表中倒数第 k 个位置上的结点 (k 为正整数)。若查找成功，算法输出该结点的 data 值，并返回 1；否则，只返回 0。要求：

(1) 描述算法的基本设计思想

(2) 描述算法的详细实现步骤

(3) 根据设计思想和实现步骤，采用程序设计语言描述算法 (使用 C 或 C++ 语言实现)，关键之处请给出简要注释。

data	link
------	------

4 算法设计与实现描述

(书上给出的基本运算外，其它问题必须先给出算法思想或步骤，再给出算法描述)

```
#ifndef LINKEDLIST_H
#define LINKEDLIST_H

#include <iostream>
// 节点定义
struct Node {
    int data;
    struct Node* next;
};
// 链表定义
struct linkedList {
    struct Node* head;
    // 构造函数：初始化 head
    linkedList() {
        head = nullptr;
    }
    // 析构函数：释放链表内存
    ~linkedList() {
        Node* current = head;
```

```

        while (current != nullptr) {
            Node* nextNode = current->next;
            delete current;
            current = nextNode;
        }
    }

    // 成员函数：基础插入（直接放到尾部）
    void insert(int value) {
        Node* newNode = new Node();
        newNode->data = value;
        newNode->next = nullptr;
        if (head == nullptr) {
            head = newNode;
        } else {
            Node* current = head;
            while (current->next != nullptr) {
                current = current->next;
            }
            current->next = newNode;
        }
    }

    // 成员函数：打印输出
    void displaylist() {
        Node* current = head;
        while (current != nullptr) {
            std::cout << current->data << " ";
            current = current->next;
        }
        std::cout << std::endl;
    }
};

#endif // LINKEDLIST_H

```

上述代码在受宏指令保护的独立头文件中，完整封装了一个基础的单向链表数据结构 `linkedList`。该结构底层由 `Node` 节点构成，每个节点被划分为存放整数的数据域和指向后继节点的指针域，而链表主体则专门负责维护一个指向首节点的 `head` 头指针。在内存生命周期管理方面，程序通过构造函数保障了初始头指针的安全置空，并利用析构函数在链表实例销毁时通过循环机制逐一释放所有节点占据的动态内存，从根源上杜绝了内存泄漏的隐患。在此核心框架之上，该结构还对外提供了用于挂载新节点的尾部 `insert` 插入函数，以及用于遍历输出有效数据的 `displaylist` 展示函数。

（1）将单链表中的奇数项和偶数项结点分解开

【算法思想】

创建两个全新的链表 `oddList` 和 `evenList`，分别作为存放奇数和偶数的容器。设置一个游标指针从原链表 `list` 的头部分开始向后逐个遍历节点。在遍历过程中，提取当前节点的数据，并判断其能否被二整除。如果能够整除，说明该数据是偶数，调用插入函数将其放进

evenList 中；反之则说明是奇数，将其放进 oddList 中。完成该节点的分配后，将游标指针向后移动一位继续处理。

【算法步骤】

- ①初始化：声明并实例化两个新链表 oddList 和 evenList 。
- ②指针定位：定义指针 current 并将其指向原始链表 list 的头部 。
- ③ 遍历与分发：利用循环结构判断 current 是否为空。若不为空，检验节点数据的奇偶性并分别调用 insert 插入对应新链表，随后 current 指针后移一位 。
- ④ 结果输出：依次调用 displaylist 函数打印原始链表、奇数链表和偶数链表 。

【算法描述和实现】

```
#include <iostream>
#include "linkedList.h"
#include <vector>
using namespace std;
void Distinguish(linkedList& list) {
    linkedList oddList; // 存储奇数的链表
    linkedList evenList; // 存储偶数的链表
    Node* current = list.head;
    while (current != nullptr) {
        if (current->data % 2 == 0) {
            evenList.insert(current->data); // 插入偶数到偶数链表
        } else {
            oddList.insert(current->data); // 插入奇数到奇数链表
        }
        current = current->next;
    }
    cout << "原始链表: ";
    list.displaylist(); // 输出原始链表
    cout << "奇数链表: ";
    oddList.displaylist(); // 输出奇数链表
    cout << "偶数链表: ";
    evenList.displaylist(); // 输出偶数链表
}
```

(2) 求两个递增有序单链表中的公共元素，放入新的单链表中

【算法思想】

建立一个新的链表 publicList 用来保存寻找出的公共节点数据。采用内外两层嵌套循环的方式进行全面比对。外层循环使用指针 i 遍历第一个链表 list1。在外层指针 i 每指向一个新节点时，内层循环开启，使用指针 j 从头到尾遍历第二个链表 list2。比对 i 和 j 所指节点的数据。如果完全一致，则确认找到交集数据，将其插入到 publicList 中并立刻打断当前的内层循环。

【算法步骤】

- ① 初始化：创建 publicList 链表，定义外层指针 i 指向 list1 的头部 。
- ② 外层遍历：在外层循环中，定义内层指针 j 指向 list2 的头部 。

③ 内层比对：在内层循环中，逐一比对 i 和 j 的数据。若相等则插入 `publicList` 并使用 `break` 提前跳出内层循环；否则 j 指针后移。

④ 外层推进与输出：内层循环结束后， i 指针后移。全部查验完毕后，判断 `publicList` 是否为空并打印相应结果。

【算法描述和实现】

```
void Publicelement(linkedList& list1, linkedList& list2) {
    linkedList publicList; // 存储公共元素的链表
    Node* i = list1.head;
    while (i != nullptr) {
        Node* j = list2.head;
        while (j != nullptr) {
            if (i->data == j->data) {
                publicList.insert(i->data); // 插入公共元素到公共链表
                break; // 找到一个匹配后跳出内层循环
            }
            j = j->next;
        }
        i = i->next;
    }
    if (publicList.head == nullptr)
    {
        cout << "没有公共元素" << endl;
    }
    else{
        cout << "公共元素链表: ";
        publicList.displaylist(); // 输出公共元素链表
    }
}
```

(3) 删除递增有序单链表中的重复元素，要求时间性能最好

【算法思想】

为追求极高的时间执行效率，采用时间开销线性的相邻节点比对法。设定一前一后两个相邻的跟踪指针 `slow` 和 `fast`，在同步向后移动的过程中持续对比两者存放的数据。一旦发现数据雷同，说明 `fast` 所指节点冗余。此时直接修改 `slow` 的连接方向使其越过 `fast`，同时利用临时指针暂存 `fast` 的地址以释放冗余内存。若数据不同，则两指针正常齐步后移。

【算法步骤】

- ①边界排查：若传入链表为空或仅有单一节点，则必定无重复，直接输出并返回。
- ②初始化双指针：指针 `slow` 锁定头节点，指针 `fast` 锁定头节点的下一个节点。
- ③步进比对：启动循环，若 `fast` 与 `slow` 数据相同，利用 `temp` 暂存 `fast` 地址，将 `slow` 指向 `fast` 的下一节点，释放 `temp` 内存，并更新 `fast` 探路。
- ④常规推进与输出：若数据不同，`slow` 和 `fast` 均向后推进一步。遍历结束后打印结果。

【算法描述和实现】

```
#include <iostream>
#include "linkedList.h"
#include <vector>
```

```

using namespace std;
void deleteElement(linkedList& list) {
    if (list.head == nullptr || list.head->next == nullptr) {
        list.displaylist();
        return;
    }
    Node* slow = list.head;
    Node* fast = list.head->next;
    while (fast != nullptr){
        if (fast->data == slow->data) {
            Node* temp = fast; // 保存要删除的节点
            slow->next = fast->next; // 删除重复元素
            fast = fast->next; // 移动快指针
            delete temp; // 释放内存
        }
        else {
            slow = slow->next; // 移动慢指针
            fast = fast->next; // 移动快指针
        }
    }
    list.displaylist(); // 输出删除重复元素后的链表
}

```

(4) 不申请新结点，合并两递增单链表并去重

【算法思想】

该算法旨在通过一次遍历完成两个递增有序链表的合并与去重，且不额外申请新的节点内存。核心思路是利用三个指针进行原地重组：指针 `p1` 和 `p2` 分别作为两个原始链表的数据探测游标，指针 `tail` 则作为新合并链表的尾部追加锚点。在同步遍历过程中，持续比较 `p1` 和 `p2` 所指节点的数据大小。每次均将数值较小的节点接入 `tail` 的后方，并使相应游标向后推进。当遇到两者数据完全相同时，仅保留 `p1` 所指的节点接入新链表，而将 `p2` 所指的冗余节点予以彻底删除释放。此方案充分利用了原链表的有序性，通过指针的重新挂载实现了极高时间性能的集合化合并。

【算法步骤】

- ① 边界安全校验：首先检查 `list1` 或 `list2` 的头指针 `head` 是否为空。若存在空指针，说明无法进行有效的双表比对，直接输出提示信息并提前结束执行。
- ② 初始化游标与尾锚点：设定指针 `p1` 指向 `list1` 的首个真实数据节点，指针 `p2` 指向 `list2` 的首个真实数据节点。同时，设定尾指针 `tail` 初始挂载于 `list1` 的头节点之上，用于后续不断向后拼接验证通过的节点。
- ③ 循环比对与重组挂载：启动循环，在 `p1` 和 `p2` 均未到达链表末尾的前提下，比对两者存放的数据大小。若 `p1` 的数据较小，将 `p1` 挂载到 `tail` 后方，并令 `tail` 和 `p1` 齐步后移；若 `p2` 的数据较小，则将 `p2` 挂载到 `tail` 后方，并令 `tail` 和 `p2` 齐步后移。
- ④ 重复节点清理：在上述循环比对中，一旦发现 `p1` 与 `p2` 的数据完全相同，则优先将 `p1` 挂载入合并链表并让 `p1` 后移。同时，利用临时指针 `temp` 暂存 `p2` 当前的地址，在 `p2` 正

常后移探路后，调用 `delete` 释放 `temp` 所占据的内存空间，从底层确保合并后的集合中不存在冗余节点。

【算法描述和实现】

```
Vvoid add(linkedList& list1, linkedList& list2) {  
    // 如果 L1 或 L2 为空，安全返回  
    if (list1.head == nullptr || list2.head == nullptr) {  
        cout<<"无公共元素";  
        return;  
    }  
    // p1 和 p2 分别指向两个链表的第一个真实节点  
    Node* p1 = list1.head->next;  
    Node* p2 = list2.head->next;  
  
    // tail 用于拼接新的合并链表，初始指向 L1 的头结点  
    Node* tail = list1.head;  
  
    // 同时遍历两个链表  
    while (p1 != nullptr && p2 != nullptr) {  
        if (p1->data < p2->data) {  
            tail->next = p1;    // 将较小的 p1 接入新链表  
            tail = p1;         // tail 指针后移  
            p1 = p1->next;     // p1 指针后移  
        }  
        else if (p1->data > p2->data) {  
            tail->next = p2;    // 将较小的 p2 接入新链表  
            tail = p2;  
            p2 = p2->next;  
        }  
        else {  
            // p1->data == p2->data 的情况（发现重复元素）  
            tail->next = p1;    // 保留 p1  
            tail = p1;  
            p1 = p1->next;  
  
            // 释放 L2 中重复的节点  
            Node* temp = p2;  
            p2 = p2->next;  
            delete temp;  
        }  
    }  
}
```

```
// 将未遍历完的剩余链表直接接到尾部  
if (p1 != nullptr) {  
    tail->next = p1;  
}
```

```

    } else if (p2 != nullptr) {
        tail->next = p2;
    }
    // 删除 L2 的头结点
    delete list2.head;
    list2.head = nullptr; // 防悬空指针
    list1.displaylist(); // 输出合并后的链表
}

```

（5）查找链表中倒数第 k 个位置上的结点

【算法思想】 采用固定间距的双指针策略，避免计算链表总长度的二次循环。初始化两个指针 `slow` 和 `fast` 均位于首个真实数据节点处。第一阶段指令 `fast` 率先独立向后行走 `k` 减一的步数。若期间触及链表末尾则说明总长度不足，直接返回失败。第二阶段令 `slow` 和 `fast` 保持既定间距同步后移。当 `fast` 抵达链表最后一个节点时，后方跟随的 `slow` 刚好停靠在倒数第 `k` 个节点上。

【算法步骤】

- ①防御检验：拦截空指针传入或 `k` 值不合法的异常调用。
- ② 前锋探路：指针 `fast` 率先利用循环独立向前移动 `k` 减一步。若中途探测到空指针则提前终止程序。
- ③ 同步滑动：在 `fast` 未达末尾的条件下，`slow` 与 `fast` 齐步向后移动。
- ④ 定位提取：循环结束时 `slow` 所在的坐标即为目标，提取数据并向系统返回成功信号。

【算法描述和实现】

```

#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* link;
};

int findKthFromEnd(Node* list, int k) {
    // 防御性编程：处理空指针或非法 k 值
    if (list == nullptr || list->link == nullptr || k <= 0) {
        return 0;
    }

    // 初始化快慢指针，指向第一个实际数据结点
    Node* slow = list->link;
    Node* fast = list->link;

```

```

    // 让 fast 指针先向前移动 k-1 步
    for (int i = 0; i < k - 1; ++i) {
        if (fast->link != nullptr) {
            fast = fast->link;
        } else {
            // fast 提前走到了链表尽头，说明链表总长度小于 k
            return 0;

```



```

    }
}

// fast 和 slow 同步向后移动

while (fast->link != nullptr) {

    slow = slow->link;

    fast = fast->link;

}

cout << slow->data << endl;

return 1;

}

```

5 运行结果截图及说明

(1)

<pre> int main() { linkedlist list; vector<int> values = {1,2,3,4,5,6,7,8,9,10,20,30,40,50,60 }; for (int value : values) { list.insert(value); } cout << "第一次输入的链表: "<< endl; Distinguish(list); list.head = nullptr; // 重置原始链表 vector<int> newValues = {10,20,30,40,50,60,70,80,90,100}; for (int value : newValues) { list.insert(value); } cout << "第二次输入的链表: "<< endl; Distinguish(list); return 0; } </pre>	第一次输入的链表： 原始链表: 1 2 3 4 5 6 7 8 9 10 20 30 40 50 60 奇数链表: 1 3 5 7 9 偶数链表: 2 4 6 8 10 20 30 40 50 60 第二次输入的链表： 原始链表: 10 20 30 40 50 60 70 80 90 100 奇数链表: 偶数链表: 10 20 30 40 50 60 70 80 90 100
---	---

说明：程序正确处理了包含连续数列和离散十位数的两个测试用例，成功按照奇偶属性将原链表分为两个独立的子链表，输出结果严格遵循预期的分组逻辑。

(2)

<pre> int main() { linkedlist list1, list2; vector<int> values1= {1,3,6,10,15,16,17,18,19,20}; vector<int> values2= {1,2,3,4,5,6,7,8,9,10,18,20,30}; for (int value : values1) { list1.insert(value); } for (int value : values2) { list2.insert(value); } cout<<"第一次实验: "<<endl; Publicelement(list1, list2); list1.head=nullptr; list2.head=nullptr; values1= {1,3,6,10,15,16,17,18,19,20}; values2= {2,4,5,7,8,9,12,22}; for (int value : values1) { list1.insert(value); } for (int value : values2) { list2.insert(value); } cout<<"第二次实验: "<<endl; Publicelement(list1, list2); list1.head=nullptr; list2.head=nullptr; values1= {}; values2= {1,2,3,4,5,6,7,8,9,10}; for (int value : values1) { list1.insert(value); } for (int value : values2) { list2.insert(value); } cout<<"第三次实验: "<<endl; Publicelement(list1, list2); return 0; } </pre>	第一次实验： 公共元素链表: 1 3 6 10 18 20 第二次实验： 没有公共元素 第三次实验： 没有公共元素
--	---

说明：程序准确提取了第一组数据中的公共节点元素。在面对第二组无交集数据及第三组含有空链表的边界条件时，程序均能精准识别并反馈“没有公共元素”，表现出良好的防错能力。

(3)

```

int main(){
    linkedList list;
    vector<int> values= {1,2,3,4,5,6,7,8,9};
    for (int value : values) {
        list.insert(value);
    }
    cout << "第一次实验: "<< endl;
    deleteElement(list);
    list.head=nullptr;
    values= {1,1,2,2,2,3,4,5,5,5,6,6,7,7,8,9};
    for (int value : values) {
        list.insert(value);
    }
    cout << "第二次实验: "<< endl;
    deleteElement(list);
    list.head=nullptr;
    values= {1,2,3,4,5,5,6,7,8,8,9,9,9,9,9};
    for (int value : values) {
        list.insert(value);
    }
    cout << "第三次实验: "<< endl;
    deleteElement(list);
    return 0;
}

```

```

第一次实验:
1 2 3 4 5 6 7 8 9
第二次实验:
1 2 3 4 5 6 7 8 9
第三次实验:
1 2 3 4 5 6 7 8 9

```

说明：对三种不同密度的冗余数据进行了去重测试。程序均能在保持原有递增顺序的前提下，铲除多余重复节点，输出序列做到了唯一化，验证了相邻比对逻辑的有效性。

(4) (5)

```

int main() {
    linkedList list1, list2;
    vector<int> values1= {1,3,6,10,15,16,17,18,19,20};
    vector<int> values2= {1,2,3,4,5,6,7,8,9,10,18,20,30};
    for (int value : values1) {
        list1.insert(value);
    }
    for (int value : values2) {
        list2.insert(value);
    }
    cout<<"第一次实验: "<<endl;
    add(list1, list2);
    list1.head=nullptr;
    list2.head=nullptr;
    values1= {1,3,6,10,15,16,17,18,19,20};
    values2= {2,4,5,7,8,9,12,22};
    for (int value : values1) {
        list1.insert(value);
    }
    for (int value : values2) {
        list2.insert(value);
    }
    cout<<"第二次实验: "<<endl;
    add(list1, list2);
    list1.head=nullptr;
    list2.head=nullptr;
    values1= {};
    values2= {1,2,3,4,5,6,7,8,9,10};
    for (int value : values1) {
        list1.insert(value);
    }
    for (int value : values2) {
        list2.insert(value);
    }
    cout<<"第三次实验: "<<endl;
    add(list1, list2);
    return 0;
}

```

```

第一次实验:
1 3 6 10 15 16 17 18 19 20 2 4 5 7 8 9 30
第二次实验:
1 3 6 10 15 16 17 18 19 20 2 4 5 7 8 9 12 22
第三次实验:
1 2 3 4 5 6 7 8 9 10

```

说明：成功完成了三组双链表的合并工作。在此过程中，不仅正确实现了首尾相连，且清空了内部的交叉重复项。第一组交集数据被合并，第二、三组也顺利完成了集合并集的构建。

6 总结、心得和建议

本次单链表实验的核心任务是基于非连续的物理内存进行数据的增删改查。相较于顺序表，单链表打破了物理内存排布的限制，节点之间仅仅依靠内部指针进行逻辑维系。这种结构赋予了极高的动态空间利用率，但也极大增加了内存调度的风险。

【心得】

(1)极高风险的内存管理挑战：在实现“合并递增单链表”任务时，由于拼接后没有及时将第二个链表的头指针置空，导致主程序在调用析构函数时对同一片内存区域进行了二次释放，直接引发底层崩溃。在删除冗余节点时，也不能仅仅是“跳过”该节点，必须严格调用释放命令将其彻底归还给系统。

(2) 巧妙的算法视角转换：在附加实验“查找倒数指定位置节点”中，最常规的二次遍历法显得冗余低效。引入“快慢双指针”策略后，让先遣指针测距，两指针保持固定间距同步滑动，彻底规避了对总长度的计算依赖，实现了单次遍历精确定位。这证明了优秀的算法设计建立在对数据结构规律深度剖析的基础之上，视角转换能带来执行效率的质变。

(3) 非连续结构对空间想象力的要求：单链表的指针断链与重连操作，要求在编写代码时必须具备极强的空间状态追踪能力，稍有偏差就会导致内存孤岛或非法越界。

【建议】

(1) 引入内存泄漏检测评估：目前实验的评价标准主要依赖终端输出的数据正确性。但在链表这种涉及频繁动态内存分配的结构中，隐性的内存不易察觉。建议在后续实验课的考核体系中引入内存泄漏检测工具，促使学生养成严密的底层编程习惯。

(2) 学习与顺序表进行对比学习，比较两者的不同之处，使理解更加深刻。